

Code Complexity Analysis Us007

Index

- Function 1: TwoDTree
 - Function 2: insertHelper
 - Function 3: inOrderTransversalData
 - Function 4: inOrderTransversalHelperData
-

1. Pseudocode

Function 1: TwoDTree

FUNCTION TwoDTree(AVL tree):

List of node datas From AVL = avl.inOrderTransversalData()
Sort by first comparator(x)

root = insertHelper(node list, depth)
END FUNCTION

Function 2: insertHelper

FUNCTION insertHelper(NodeData list, depth):

IF list is NULL || List is over
END

Increment size
Sort List according to depth(x or y)

Get midian index and create this node

split the list in half(using median index)

node.setLeft (insertHelper(leftSubList, depth + 1))
node.setRight (insertHelper(rightSubList, depth + 1))

END FUNCTION

Function 3: inOrderTransversalData

FUNCTION inOrderTransversalData():

Start list of nodes

```

call helper(inOrderTransversalHelperData(list, tree_root))
END FUNCTION

```

Function 4: inOrderTransversalHelperData

```

Functions inOrderTransversalHelperData(List nodes, Node node):
    IF node is NULL
        END

        inOrderTransversalHelperData(List, left node)
        nodes.add(node element)
        inOrderTransversalHelperData(List, Right node)
END FUNCTION

```

3. Time Complexity

Table 1: TwoDTree Function

Operation	Description	Best Case	Average Case	Worst Case	Notes
inOrderTransversalHelperData	Extract all nodes from AVL	$O(n)$	$O(n)$	$O(n)$	Linear traversal
insertHelper	Build 2D tree	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	One visit per element
TOTAL	Complete tree construction	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Sorting dominates

Table 2: insertHelper Function

Operation	Description	Best Case	Average Case	Worst Case	Notes
Base case	Empty list check	$O(1)$	$O(1)$	$O(1)$	Stops recursion
Sorting list	Sort by selected axis	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Happens at each recursive level

Operation	Description	Best Case	Average Case	Worst Case	Notes
Build node	Create median node	$O(1)$	$O(1)$	$O(1)$	Constant
Split list	Partition into halves	$O(n)$	$O(n)$	$O(n)$	Dependent on list size
Recursive calls	Two subtree constructions	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Height $\times$ sort cost
TOTAL	Recursive kd-tree building	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	

Table 3: inOrderTransversalData Function

Operation	Description	Best	Avg	Worst	Notes
Init list	Create empty list	$O(1)$	$O(1)$	$O(1)$	Constant
Call helper	Traverse entire AVL	$O(n)$	$O(n)$	$O(n)$	Visits all nodes
Return list	Return result	$O(1)$	$O(1)$	$O(1)$	Constant
TOTAL	Retrieve AVL sorted data	$O(n)$	$O(n)$	$O(n)$	Linear time

Table 4: inOrderTransversalHelperData Function

Operation	Description	Best	Avg	Worst	Notes
Null check	Base case	$O(1)$	$O(1)$	$O(1)$	Constant
Left recursion	Visit left subtree	$O(n)$	$O(n)$	$O(n)$	Part of full traversal
Add element	Insert node data	$O(1)$	$O(1)$	$O(1)$	Constant
Right recursion	Visit right subtree	$O(n)$	$O(n)$	$O(n)$	Part of full traversal
TOTAL	Full inorder traversal	$O(n)$	$O(n)$	$O(n)$	Every node visited once